

POR FIN LLEGO EL MOMENTO DEL INICIO:

Debemos pensar de la forma en que este mundo de la programación se basa siempre en solicitudes $\leftrightarrow$ respuestas, es decir nosotros para poder exponer un servicio o un video para dar un ejemplo mas preciso y que mucha gente o cierta cantidad de gente pueda ver ese video que nosotros queremos compartir debemos subir ese video a un servidor o simplemente exponer nuestra pc local (que no es buena idea para muchas personas que lo quieran ver) y con la url que obtengamos permite a personas (computadoras, teléfonos, dispositivos externos) conectarse al servidor enviando una solicitud de que quieren ver el video que se encuentra dentro de él y el servidor da una respuesta para devolver el video al dispositivo que lo está solicitando

Ahora respondemos algo que siempre tuve duda y sobre la cual no había buscado, para que sirven los navegadores como GOOGLE, Internet Explorer, Mozilla Firefox, OperGX, Brave... Para ello comprendí algo, si bien una música necesita un reproductor de música para escucharse, si bien un video necesita un reproductor de video para verse.... debemos tener en cuenta que un código de programación ya sea html ... necesita una herramienta que lea ese formato de códigos para nosotros poder ver el contenido, eso es un navegador web, un lector de formatos de código

HTML (HYPERTEXT MARKUP LANGUAGE) LENGUAJE DE MARCADO DE HIPERTEXTO: **Para poner comentarios en html es `<!-- -->`**

Es un lenguaje de marcado, es decir, e etiquetas, en donde lo que hacemos netamente es estructurar nuestra arquitectura o nuestra aplicación, puro bloques de texto, inserción de imágenes, todo ello ya que html nos permite interpretar lo que nosotros queremos mostrar, pero con salidas más impecables. Eso es html estructuras o bloques de texto que forman la carcasa de nuestro frontend

CSS (CASCADING STYLE SHEETS) HOJA DE ESTILO EN CASCADA **Para poner comentarios en css es `/* */`**

Es un lenguaje para personalizar las propiedades del lenguaje marcado, como lo es el html, para entender esto debemos situarnos a años atrás, en donde como ya dije anteriormente cada navegador es una herramienta de reproducción del código que nosotros desarrollamos, que pasa con eso?, que cada navegador viene con una serie de propiedades personalizadas, que cuando tu por ejemplo escribes un `<h1>hola</h1>` que es el máximo texto de encabezado en html, cada navegador tenía una configuración distinta, es decir, por ejemplo el navegador google le ese h1 y decía “aaa yo a este h1 tengo configurado que cuando lo vea para mostrarlo tengo que poner 16mp de tamaño extra, tengo que ponerle un borde negro....” Y así cada navegador distinto tenia configuraciones distintas, que al momento de que alguien viese el mismo software en distinto navegadores lo iba a ver con diferencias o simplemente habría que crear códigos distintos para cada navegador debido a que si un navegador tenia la propiedad de sombras para los textos y otro no, pues entonces había que crear código distinto para cada navegador, lo cual es inviable, por ello nacio css para ser un lenguaje general como de

personalización y lo que tu veas en un navegador sea lo mismo que veas en otro navegador.

HTML Y CSS JUNTOS

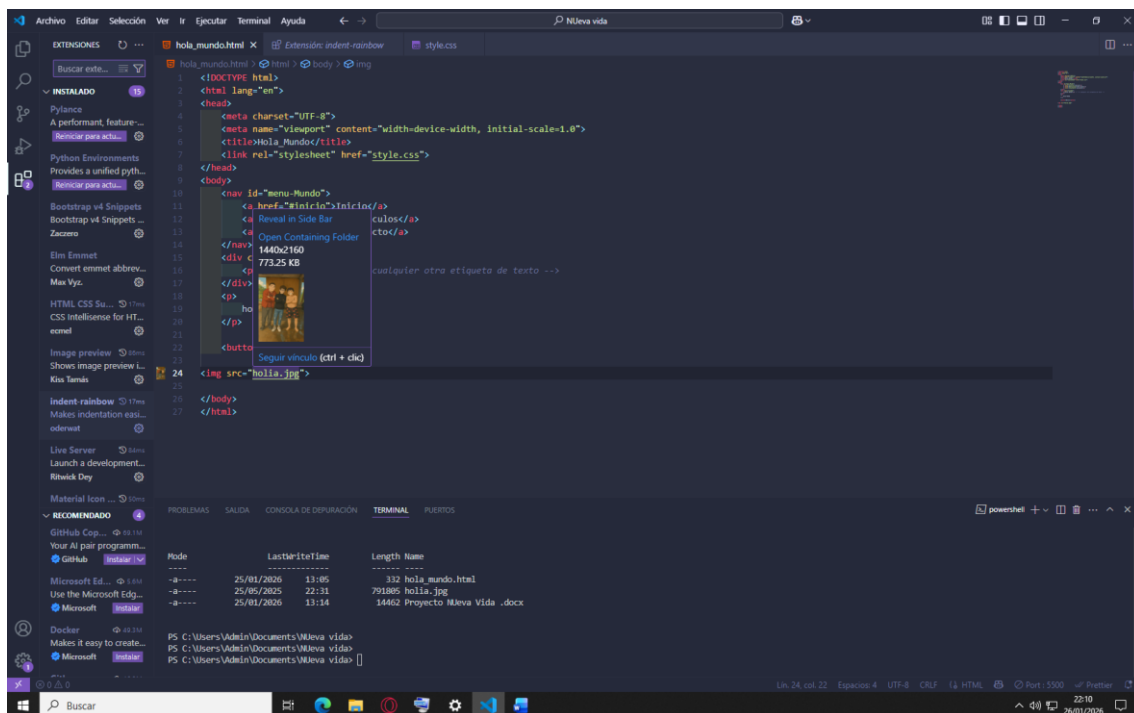
En conclusión, HTML y CSS se complementan, ya que HTML crea estructuras y elementos y CSS agarra esas estructuras y elementos y les modifica todas sus propiedades para que el navegador muestre lo que queremos exactamente y no lo predeterminado que el trae.

HERRAMIENTAS DE DESARROLLO DE CÓDIGO:

Debemos tener en cuenta que para programar y evitar el exceso de errores humanos los desarrolladores optan por usar software de desarrollo de código, ya que vienen con complementos que nos ayudan a agilizar el trabajo y más allá de eso evitar errores, porque imaginemos nos programar en un block de nota jaja. Por ello inclusive hay software denominado ide (entorno de desarrollo integrado) que es un software de desarrollo de código, pero con extensiones y mas funcionalidades

Para este curso vamos a hacer uso de vscode, unas extensiones muy curiosas de este ide y que ayudan mucho son:

- **Image preview:** nos ayuda al posicionar encima de una imagen insertada en `img src` nos muestra la imagen desde la miniatura, si no la muestra es porque esta mal insertada:



- **indent-rainbow:** nos muestra columnas de cada color para saber donde esta ubicada la identacion de algo que quedo por arriba votado

- **Live Server:** Para ver cambios en vivo al recién guardar en el archivo se reflejan en el navegador sin tener que recargar el navegador
- **Palenight Theme:** tema de color más bonito y de agrado en los textos programáticos que vamos desarrollando.

Un dato curioso es que los navegadores web como Google opera.... Desde sus inicios de los servidores web en 1990 buscaban el archivo index.html en la ruta en donde tuviesen parado, esto era para facilitar la URL ,as limpia y ordenada así como una búsqueda más fácil y flexible del archivo índice o raíz de nuestro proyecto, esta etimología a día de hoy se sigue usando, podemos hacer pruebas creando una carpeta, y dentro de ella varios archivos de nombres variados.html y uno de ellos como index.html, al abrir eso en el navegador se nos abrirá automáticamente el index.html jajaja, ahora nos preguntaremos ¿Cómo hacemos para hacer estas prácticas ya que mi pc local no es un servidor web?, se puede con un modulo de Python muy utilizado que lo que hace es exponer tu pc o servirla predeterminadamente al puerto 8000, obviamente debemos dirigirnos a la ruta en donde esta nuestra carpeta y dentro de ella hacemos el siguiente comando:

- `python -m http.server`

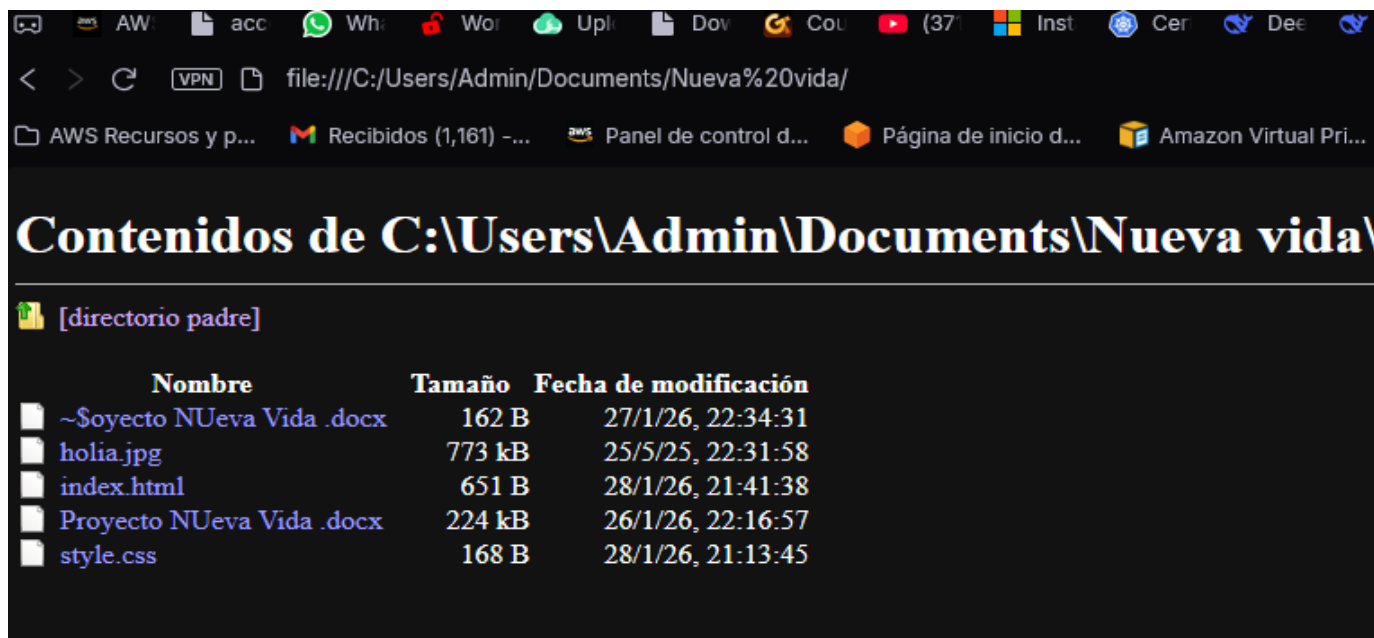


```
PS C:\Users\Admin\Documents\Nueva vida> python -m http.server
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
::1 - - [28/Jan/2026 21:48:34] "GET / HTTP/1.1" 200 -
::1 - - [28/Jan/2026 21:48:34] "GET /style.css HTTP/1.1" 304 -
::1 - - [28/Jan/2026 21:48:46] "GET // HTTP/1.1" 200 -
::1 - - [28/Jan/2026 21:48:46] "GET //style.css HTTP/1.1" 200 -
::1 - - [28/Jan/2026 21:48:46] "GET //holia.jpg HTTP/1.1" 200 -
```

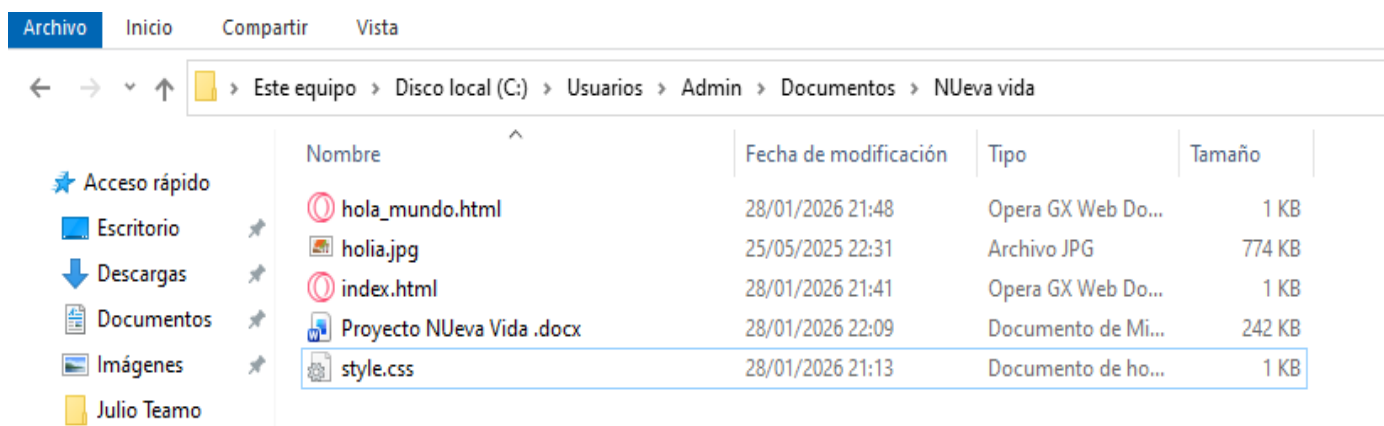
Nos vamos a nuestro navegador y colocamos <http://localhost:8000> y de una si tenemos un archivo index.html en esa primera carpeta de nuestro proyecto nos cargara automáticamente ese código, esto es lo mas cercano a simular un servidor web local de manera sencilla y muy eficiente, casi asemejándose a producción en empresas, pero obviamente mucho más sencillo de replicar gracias a Python

Hay muchas formas mas de hacer de nuestra pc un servidor web local, mas sin embargo como dije anteriormente la mejor es con Python, hay otra forma, pero lo único malo es que no nos busca los archivos index.html automáticamente, pero es utilísima para cuando no tenemos un XAMPP o Python y queremos ver la estructura de nuestro proyecto por un navegador, simplemente seria irse a la ruta donde esta la carpeta de nuestro proyecto, la URL que pondríamos en el navegador seria así:

- <file:///C:/Users/Admin/Documents/Nueva%20vida/>



Esto es dentro de opera jaja, lamentablemente no nos abre automáticamente el index.html pero es bueno conocer esta forma también, cabe resaltar que aunque en Windows lo tengamos en español el sigue utilizando en los navegadores de fondo el inglés, me paso ya que toda esa ruta de la URL yo la tenía en español así:



Pero cuando la ponía en español en el navegador no servía jaja no encontraba mi pc jajaja, entonces dejo ese dato curioso, así como que la carpeta Nueva vida tiene un %20 en medio y eso se debe a que en Windows se escapa caracteres especiales con un porcentaje y seguido de el numero decimal de ese carácter, o simplemente poniendo entre paréntesis jajajaja también sirve, bueno eso dice la documentación, pero a mi no me sirvió, algo puse mal supongo.

Ahora si continuando con html algo que es super curiosos de tener en cuenta es que no se puede programar por programar, ósea enredar tu código con etiquetas mal usadas que si negritas a un texto dentro del html y tratar de ponerle estilos allí con etiquetas de html, ya que para eso esta css, además ese viaje de etiquetas o el mal uso de html nos perjudica en algo que es lo que mas me llamo la atención, y es el echo de que cuanto mejor sea tu código y mas entendible para un navegador

como Google tendrás una posición mas arriba en la búsqueda, es decir, cuando una persona busque algo relativo a tu pagina web si tu código es bien entendido por Google te pondrá mas cerca de los primeros resultados que ve la persona o usuario, eso es algo importantísimo a tener en cuenta

ESTRUCTURA DE UNA PÁGINA HTML.

Una página o documento desarrollado en html consta de 4 apartados importantes, un DOCTYPE, este sirve para **Especifica la versión de HTML** Le dice al navegador que el documento está escrito en **HTML5** (la versión actual y moderna de HTML). **Activa el "modo estándar" (standards mode)** Sin esta declaración, los navegadores pueden entrar en "modo quirks" (quirks mode), que emula comportamientos antiguos y puede causar problemas de visualización. Cabe destacar que en versiones anteriores de html si se utiliza igualmente esta etiqueta, pero es distinta, por ejemplo, para:

HTML 4.01 (versión anterior):

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
```

XHTML 1.0:

html

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

Ya en html5 se resumió así de sencilla como lo dijimos arriba, por otro lado también podemos encontrar dentro den código html la etiqueta <html> puesto es la que indica que todo lo que este dentro de ella es parte de la página, luego encontramos la etiqueta <head> en donde encontramos información que no podemos ver a simple vista, bueno no totalmente, ya que aquí ponemos todos los metacaracteres de la página como título de la ventana.... Ya por ultimo encontramos el <body> que es todo lo que va a aparecer en la pagina web, no en la parte superior de la ventana ya que es del head, sino en el cuerpo de toda la página...

Así se ve la estructura de lo mencionado anteriormente en un editor de código:

```
<!DOCTYPE html>
<html>

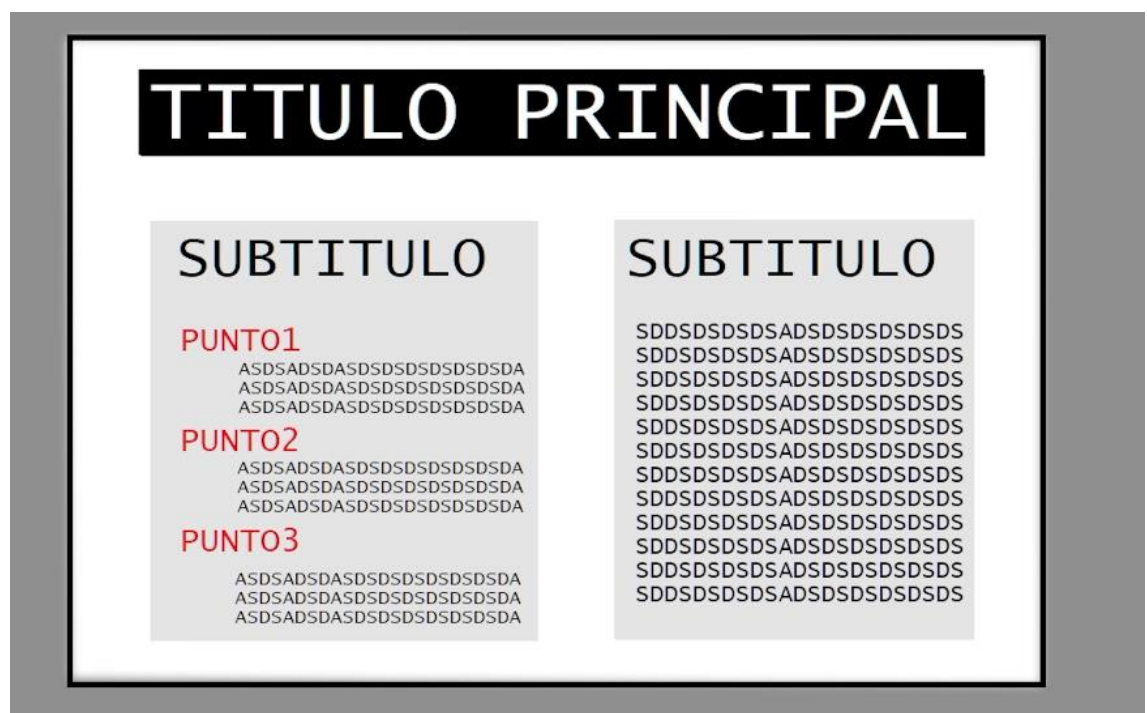
  <head>
    <title>HEADpag</title>
  </head>

  <body>
    daniel
  </body>

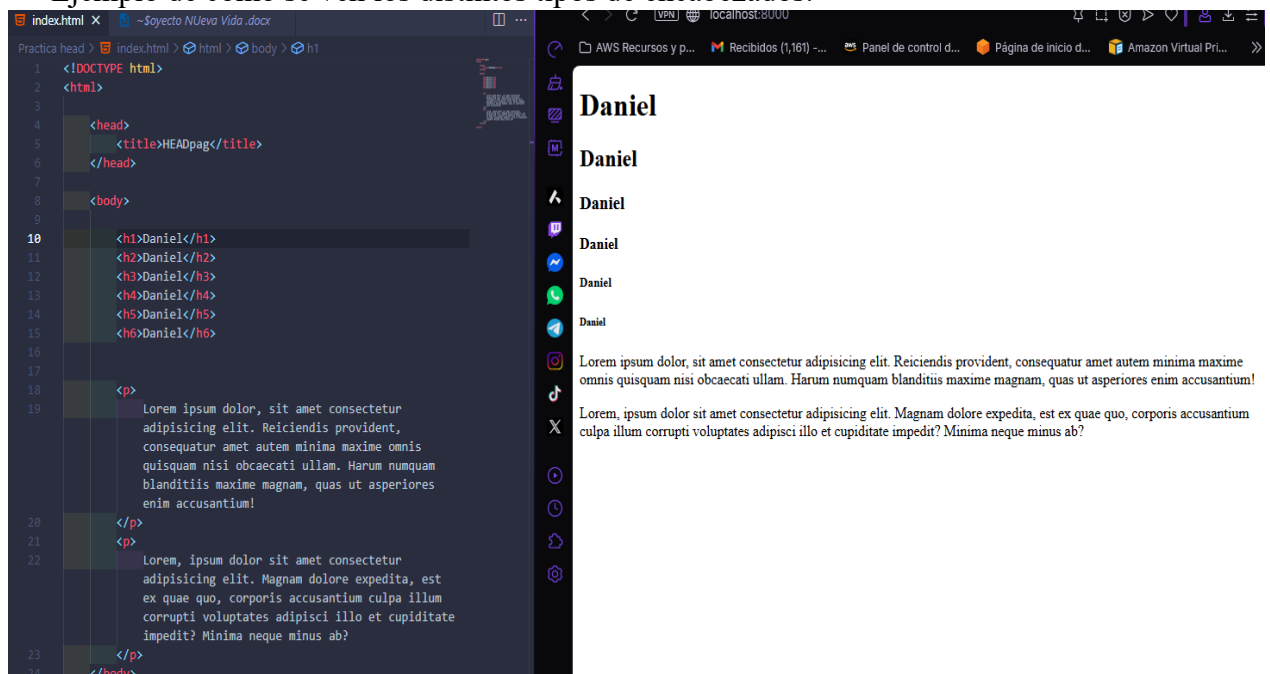
</html>
```

ENCABEZADOS:

Algo muy importante de denotar es que el h1 ósea los encabezados son una de las etiquetas más importantes, ya que Google o el motor de los navegadores se basan en leer el h1 que es el encabezados para el titulo mas importante del tema de la página, lo leen y ya saben de qué va la página, mas sin embargo si la gente juega a poner más h1 pues Google no mostrara mucho tu página en el navegador ya que es una infracción a las reglas estándares de programación, un ejemplo es las personas que son ciegas y cuyos aparatos les van transcribiendo la página, esos aparatos leen el h1 y se pierden si ven más de un h1 ya que no sabrán con certeza de que va la página, por ello el estándar es un h1 por apartado, es decir, por cada archivo que nos lleve a otra pagina un solo h1 por cada uno, Ejemplo de cómo deb3eria ser el orden de encabezados:

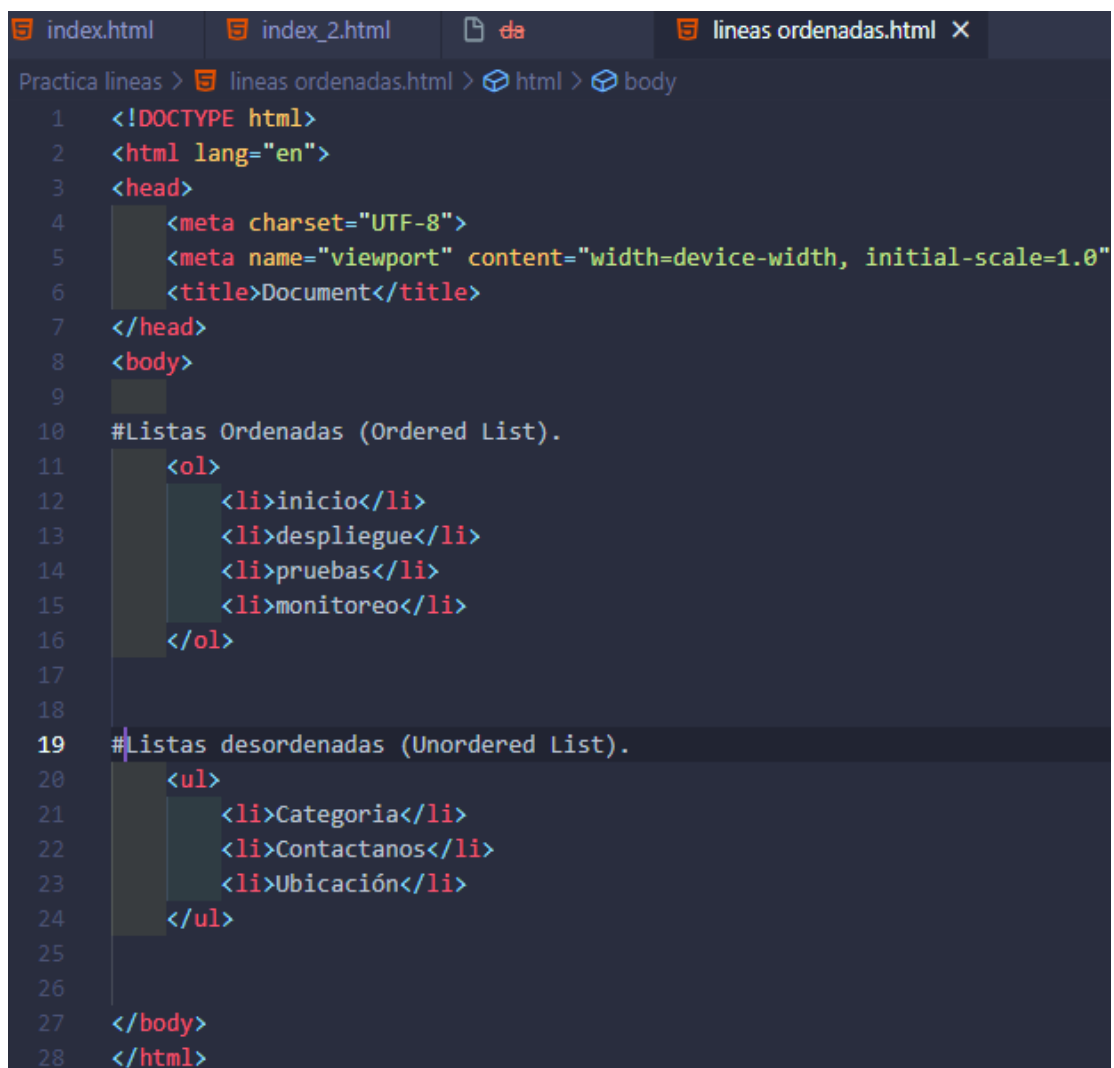


Ejemplo de como se ven los distintos tipos de encabezados:



LISTAS

Por otro lado también tenemos las listas, algo muy importante en html ya que ellas suelen utilizarse mucho en navbar o pasos a seguir en apartados específicos, tenemos las listas ordenadas (ordered list) y las desordenadas (unordered list), diremos para que coño cada una jajajajua uno puede utilizar como se le de la gana lo que debemo9s tener en cuenta es que todo va de una buena sintaxis si quieres ser buen programador, por ello debemos utilizar las listas ordenadas en apartados que lleven un orden de pasos, y las listas desordenadas en donde la consecuencia no sea estricta, por ejemplo, en el navbar ya que el usuario es libre de meterse en las categorías, o de ir al carrito de compra..... no necesita un orden de ejecución, todo es a la disposición de lo que quiera hacer, en html se representan así:



```
index.html index_2.html index_3.html lineas ordenadas.html X
Practica lineas > lineas ordenadas.html > html > body
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9
10 #Listas Ordenadas (Ordered List).
11 <ol>
12   <li>inicio</li>
13   <li>despliegue</li>
14   <li>pruebas</li>
15   <li>monitoreo</li>
16 </ol>
17
18
19 #Listas desordenadas (Unordered List).
20 <ul>
21   <li>Categoria</li>
22   <li>Contactanos</li>
23   <li>Ubicación</li>
24 </ul>
25
26
27 </body>
28 </html>
```

Otra cosa a tener en cuenta antes de seguir avanzando es que, una pagina web es solamente un archivo html nada mas así como lo estamos viendo en los ejemplos de arriba, pero un sitio web es un conjunto de páginas web

ENLACES:

Los enlaces nos llevan ya sea a una pagina por la web o a un archivo local, para ir a una web externa debemos poner en el `<a href="https://youtube.com.">` ya que si solo colocamos `<a href="youtube.com">` sin ponerle el `https://` el navegador sobreentenderá que es un archivo local, de igual manera para la versión actual de html5 ya no es necesario poner el `https://` con puro poner `<a href="//youtube.com">` ya basta, pero por buena lógica programacional es bueno seguir utilizando el `https` por decencia jajaja, de otra forma es importante denotar que cuando tu utilizas un enlaces puedes hacer que al clickear sobre el se abra en una ventana nueva o encima de la ventana sobre la que nos encontramos, estos son los conocidos atributos de html, son iguales que las propiedades de css pero son atributos únicos de html que no tiene css, por ello son atributos de html, debemos recordar que dependiendo de la etiqueta puede o no puede que tenga atributos, por ejemplo para los hipervínculos que es lo mismo que los enlaces ancla links Hay atributos como por ejemplo el de hacer que el vinculo de abra en una nueva ventana o encima de la que nos encontramos parados, cabe destacar que por defecto si no definimos este atributo el html conjunto con el navegador utilizara por defecto `"_self"` que es abrir por encima, igual aquí la explicación de cada target:

El atributo target en HTML define dónde se abrirá el documento enlazado al hacer clic en un hipervínculo (`<a>`). Sus valores principales son `_self` (mismo marco/pestaña, predeterminado) y `_blank` (nueva pestaña o ventana). Se usa para gestionar la navegación sin perder el contexto del sitio original.

Valores principales del atributo target:

- **`_self`:** Valor por defecto. Abre el enlace en la misma pestaña o ventana actual.
- **`_blank`:** Abre el enlace en una nueva pestaña o ventana. Muy común para enlaces externos para que el usuario no abandone el sitio web original.
- **`_parent`:** Abre el enlace en el marco padre (si no hay marcos, funciona igual que `_self`).
- **`_top`:** Rompe todos los marcos y carga el sitio en toda la ventana del navegador.

Después los veremos con más atención 😊

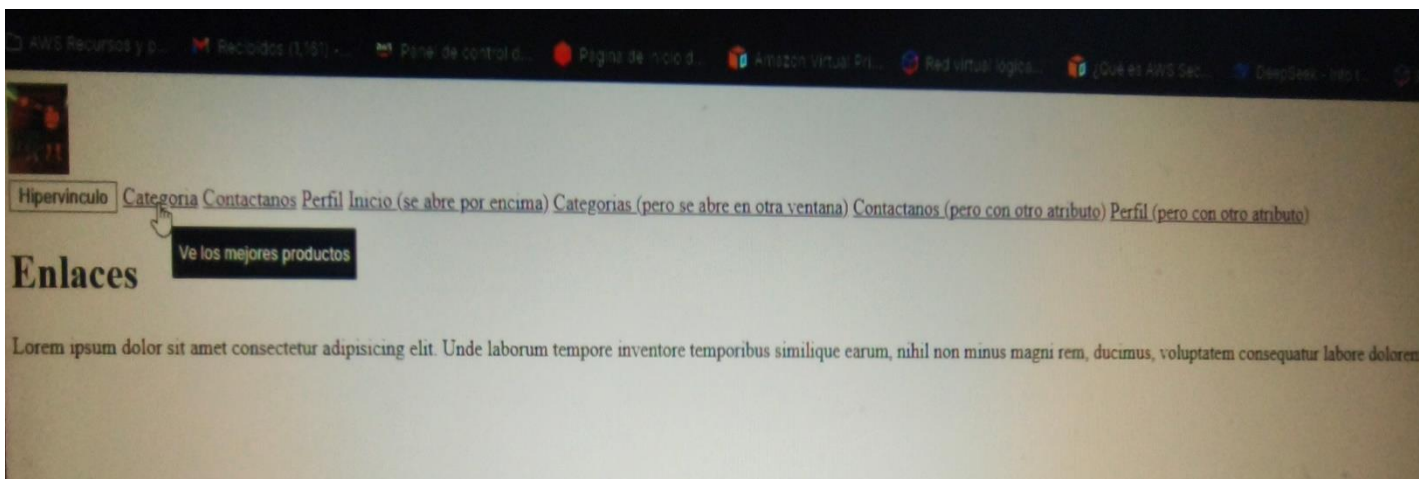
ESTRUCTURA DE LOS ENLACES:

```
index.html  enlaces.html X  enlace1.html  enlace2.html  enlace3.html  index_2.html  da  lineas ordenadas.html

Practica Enlaces Andas links > enlaces.html > html > body
2  <html lang="en">
3  <head>
7  </head>
8  <body>
9
10  <div class="logo">
11    <a href="enlaces.html">
12      
13    </a>
14  </div>
15
16  <div class="navbar">
17    <a href="https://www.youtube.com/watch?v=ELSm-G201Ls"><button>Hipervinculo</button></a>
18    <a href="enlace1.html">Categoria</a>
19    <a href="enlace2.html">Contactanos</a>
20    <a href="enlace3.html">Perfil</a>
21    <a href="enlaces.html" target="_self">Inicio (se abre por encima)</a>
22    <a href="enlace1.html" target="_blank">Categorias (pero se abre en otra ventana)</a>
23    <a href="enlace2.html" target="_parent">Contactanos (pero con otro atributo)</a>
24    <a href="enlace3.html" target="_top">Perfil (pero con otro atributo)</a>
25  </div>
26
27  <h1>Enlaces</h1>
28
29  <p>
30    Lorem ipsum dolor sit amet consectetur adipisicing elit. Unde laborum tempore inventore temporibus similique earum, nihil non minus magni rem, ducimus,
31    voluptatem consequatur labore dolore nemo quod ad officiis eveniet?
32  </p>
33 </body>
</html>
```

Otra cosa interesante es que hay otro atributo que no se si es solo de los hipervínculos, ya lo probare jaja, que se llama `title=""` literalmente tu pones un comentario y cada que pases por encima el mouse de ese hipervínculo se mostrara esa descripción jaja

```
<div class="navbar">
  <a href="https://www.youtube.com/watch?v=ELSm-G201Ls"><button>Hipervinculo</button></a>
  <a href="enlace1.html" title="Ve los mejores productos">Categoria</a>
```



Cabe destacar que este atributo title ya vi y si se puede utilizar sobre casi todo jajaja, pero obviamente por buenas prácticas es bueno utilizarlo solo en imágenes para dar una descripción o en hipervínculos para dar una información de adonde llega ese vínculo

SELF CLOSING TAGS: (ETIQUETAS QUE SE CIERRAN AUTOSOLAS SIN TENER QUE CERRARLAS CON UN </>)

IMÁGENES:

Etiquetas de bloques vs Etiquetas de líneas

La etiqueta para ingresar imágenes en html no es necesario cerrarla, ósea, hacer lo siguiente está mal:

- `</img>` (Así está mal puesta)
- `` (Así esta bien puesta)

Por otro lado, debemos saber que ese atributo “src=” ” ” su acrónimo es “source” en inglés, cuyo significado en español es “Fuente”

Otra cosa importante es que la etiqueta imagen tienen un atributo “alt= “ ” ” el cual es por si la imagen no carga ya sea que se borro del servidor o ya no existe pues ese atributo “alt (alternative)” nos mostrara una descripción con lo que pongamos (debería ser una descripción de que es la foto), cabe destacar que ese texto solo se mostrara si la imagen no se ve ósea que no cargue, ya que cuando carga bien ese texto de “alt” no cargara

Otra cosa interesante de ese atributo “alt” es que si lo definimos ósea si siempre lo colocamos eso ayuda a que el motor de búsqueda de Google recomiende esa imagen dentro del apartado de imágenes del navegador cuando alguien busque una imagen relacionada con la descripción que establezcamos dentro del alt

RUTAS (ROUTE)

Debemos tener en cuenta que al momento de trabajar con html, css, javascript... e infinidad de lenguajes de programación e incluso sistemas operativos como lo es en linux tenemos rutas, por ejemplo en Linux si queremos acceder a una ruta del SO podemos hacer un `cd /home/usuario.....` y así accedemos a las rutas, en el mundo de la programación no es distinto a ello ya que por ejemplo si hablamos a nivel de html para acceder a una imagen y mostrarla como vimos en el apartado anterior pero dicha imagen se encuentra en otra carpeta que no es donde esta el archivo que estamos programando y el cual es donde vamos a insertar la imagen debemos poner la ruta completa de la imagen, por ejemplo:

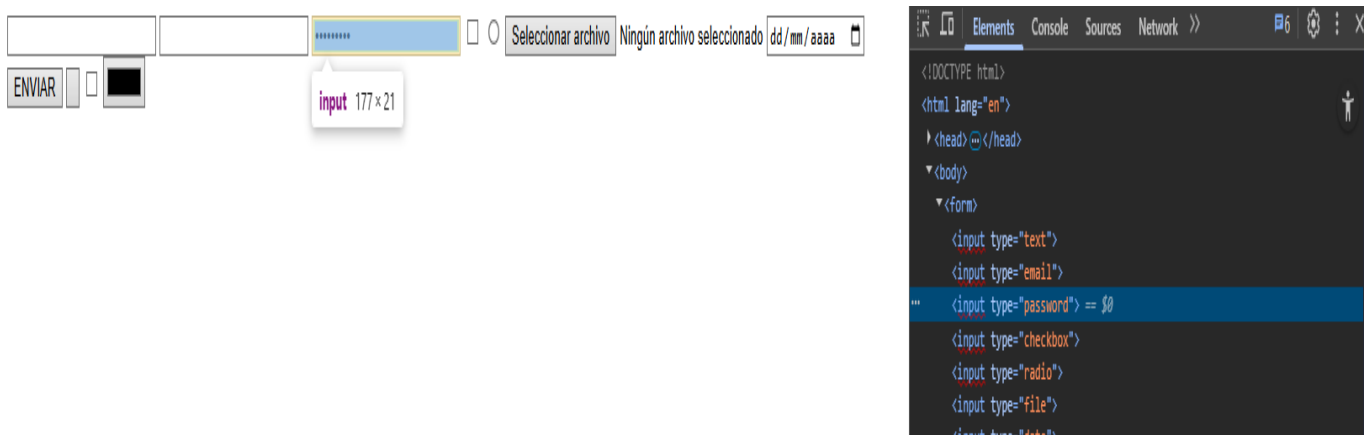
- Si la imagen se encuentra una carpeta atrás:
``
- Si la imagen esta una carpeta atrás pero dentro de otra carpeta:
``

FORMULARIOS

Un formulario es una capa de las más importante en la programación ya que los formularios se basan en entradas de información en base a una pregunta o verificación previa, es decir, es la forma en que obtenemos datos de un usuario o cualquier tipo de información, cabe destacar que hay muchos tipos de input como pedir el nombre de un usuario, apellido, mail bajo un formato de mail, una barrita en donde seleccione la edad.... Todo ellos son entradas de información que aunque tenga distintas formas de pedirla no cambia el echo de que un formulario es una cadena de texto ...



Podemos observar como se ven los diferentes tipos de entradas de información que se pueden realizar con los forms (formularios), un tip importante y que es un truquito de programación y se aplica a todo porque lo comprobé con varias paginas incluso la de Instagram es que si tu le das inspeccionar condigo a la página de Google y donde está el formulario `input type="password"` si lo cambiamos editamos en vivo sobre ese inspeccionar en el navegador podemos ponerlo tipo text y vemos la contraseña que se ve con asteriscos, obviamente esto es útil para cuando la página web no tiene el icono de visibilidad de la contraseña, ejemplo:



Si lo modificamos saldría así:

DANIELLLL ☐ ☐ Seleccionar archivo Ningún archivo seleccionado dd/mm/aaaa

ENVIAR ☐ ☐

```
<!DOCTYPE html>
<html lang="en">
  <head>
  </head>
  <body>
    <form>
      <input type="text">
      <input type="email">
      <input type="text"> == $0
      <input type="checkbox">
      <input type="radio">
      <input type="file">
      <input type="date">
      <input type="submit" value="ENVIAR">
```

Eso se aplica igual para cualquier pagina de internet, miremos la de Instagram:



Mira los momentos cotidianos
de tus mejores amigos.

Iniciar sesión en Instagram

Iniciar sesión

¿Olvidaste tu contraseña?

```
<input class="x1i10hfl xggy1 nq xtpw4lu x1tutvks x1s3xk63 x1s07b3s x1a2a7pz xjbqb8w x1v8p93f x1o3j0l2 x16stqj xv5 lvn5 x1ejq3ln x18oe1m7 x1sy0 etr xstzfhl x972fbf x10w94by x1qhh985 x14e42zd x9f619 xzs f02u x1l1i1h x15h3p50 x10em q54 x1vr9vpq x1iyjqo2 x10d0g m4 x1fhayk4 x16wd1z0 x3cjxhe xe9ewy2 x1llt19s xeugli xly ipyv x1hcrkkg xfvqzld x12vv8 92 x1hu168l x1ttzon8 x1sfh74k x3fqe8q x185fvkj x1p97g3g xm tqnhx x1liq0mb xgm6d7 x1quw 8ve xx0ingd xdj266r xyysdx x14vy60q x109j2v6 xp5op4 xly 44fgy xdzva22 xs8nz04 x1fzeh xr xha3pab" dir="ltr" aria-invalid="false" id="R_33d91plcdcpbn6b5ipamH1_1" type="password" name="pass" value="DANIELLLL">
```

Y así queda cuando le ponemos 'text'



Mira los momentos cotidianos
de tus mejores amigos.

Iniciar sesión en Instagram

Iniciar sesión

¿Olvidaste tu contraseña?

```
<input class="x1i10hfl xggy1 nq xtpw4lu x1tutvks x1s3xk63 x1s07b3s x1a2a7pz xjbqb8w x1v8p93f x1o3j0l2 x16stqj xv5 lvn5 x1ejq3ln x18oe1m7 x1sy0 etr xstzfhl x972fbf x10w94by x1qhh985 x14e42zd x9f619 xzs f02u x1l1i1h x15h3p50 x10em q54 x1vr9vpq x1iyjqo2 x10d0g m4 x1fhayk4 x16wd1z0 x3cjxhe xe9ewy2 x1llt19s xeugli xly ipyv x1hcrkkg xfvqzld x12vv8 92 x1hu168l x1ttzon8 x1sfh74k x3fqe8q x185fvkj x1p97g3g xm tqnhx x1liq0mb xgm6d7 x1quw 8ve xx0ingd xdj266r xyysdx x14vy60q x109j2v6 xp5op4 xly 44fgy xdzva22 xs8nz04 x1fzeh xr xha3pab" dir="ltr" aria-invalid="false" id="R_33d91plcdcpbn6b5ipamH1_1" type="text" name="pass" value="DANIELLLL">
```

Algo importante es que para los input de los formularios hay 4 atributos muy importantes:

- required= este atributo hace que el input sea obligatorio, es decir, que cuando nosotros le demos al botón enviar si no pusimos el texto en ese input o no seleccionamos el archivo o la fecha... dependiendo del tipo de input no nos va a dejar avanzar, nos saldrá una casilla bien bonita en donde nos pedirá que coloquemos el texto:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Formulario</title>
7 </head>
8 <body>
9   <form>
10    <input type="text" required>
11    <input type="email">
12    <input type="password">
13    <input type="checkbox">
14    <input type="radio">
15    <input type="file">
16    <input type="date" required>
17    <input type="submit" value="ENVIAR">
18    <input type="button">
19    <input type="checkbox" name="daniel" id="12345">
20    <input type="color">
21    <input type="file"> </input:file>
22  </form>
23
24  <a href="https://youtube.com">youtube</a>
25
26 </body>
27 </html>
```

Así lo vemos en la web:

Completa este campo

- name= este atributo nos permite que el input o mejor dicho la información que está poniendo el usuario cuando le de enviar esa información quede referenciada en una cajita y cuyo identificador va a ser el nombre que le pongamos, es decir, luego para hacer uso de esa información que si validar en la base de datos.... Hacemos el llamado a ese name ya que contiene esa información._

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fourmulario</title>
</head>
<body>
  <form>
    <input type="text" required name="textodaniel">
    <input type="email">
    <input type="password">
    <input type="checkbox">
    <input type="radio">
    <input type="file">
    <input type="date">
    <input type="submit" value="ENVIAR">
    <input type="button">
    <input type="checkbox" name="daniel" id="12345">
    <input type="color">
    <INput:file> </INput:file>
  </form>
  <a href="https://youtube.com">youtbe</a>
</body>
</html>
```

Así se ve en la web:

☐

☐

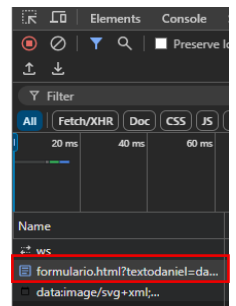
Seleccionar archivo

Ningún archivo seleccionado

dd/mm/aaaa

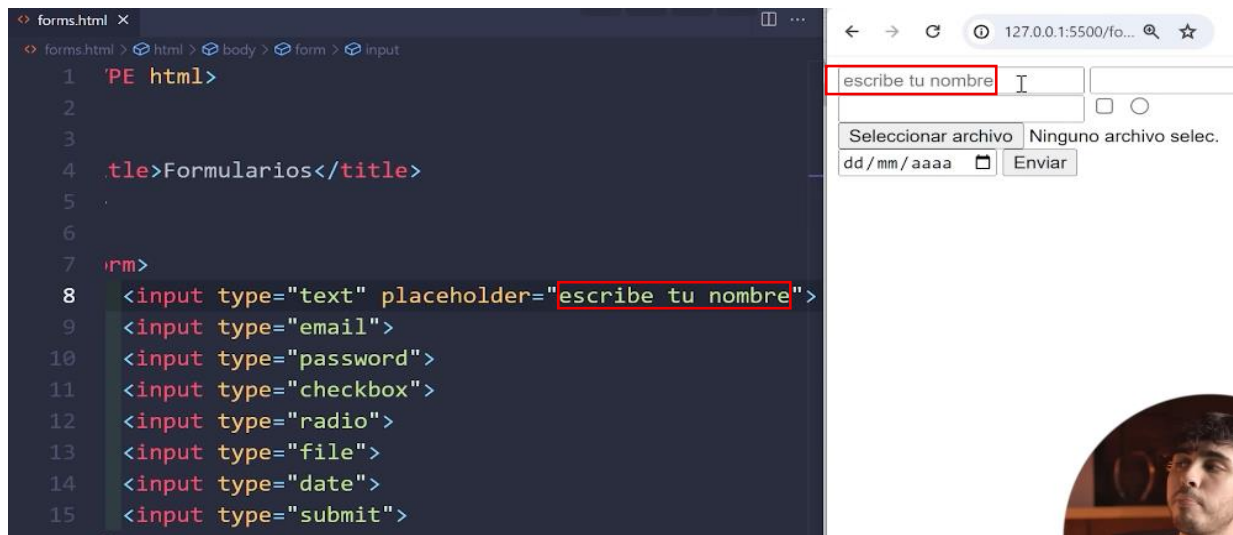
ENVIAR

youtube

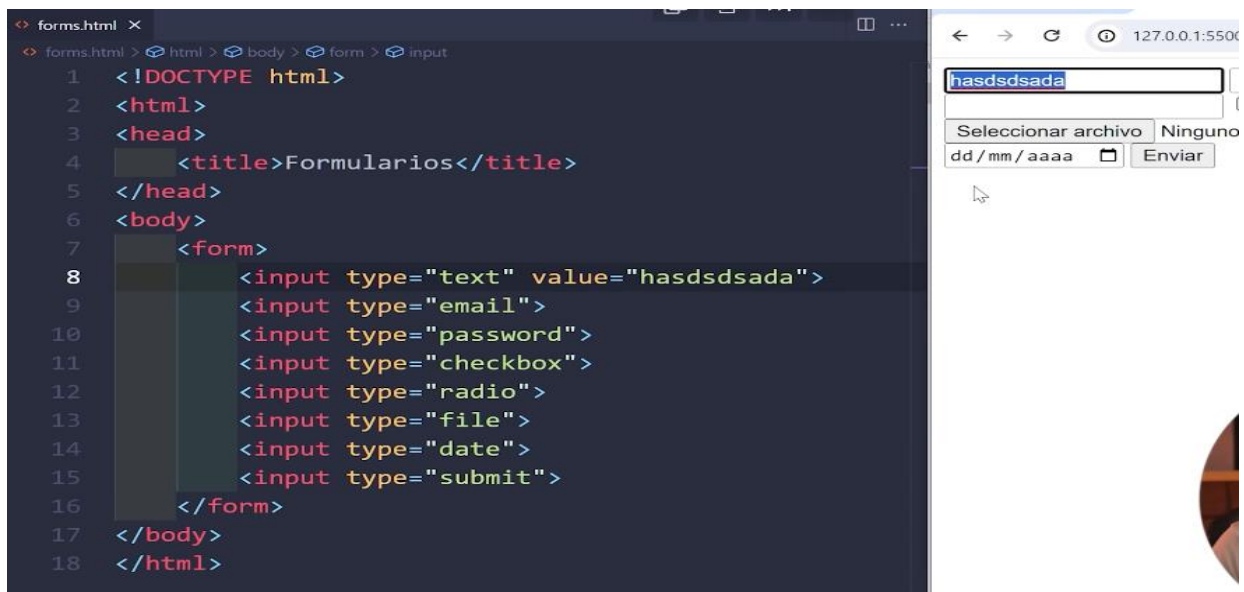


Posdemos ver que ahora en la inspeccion sale que hay algo guardado y sale el identificador que puse en el html “name=”textodaniel”

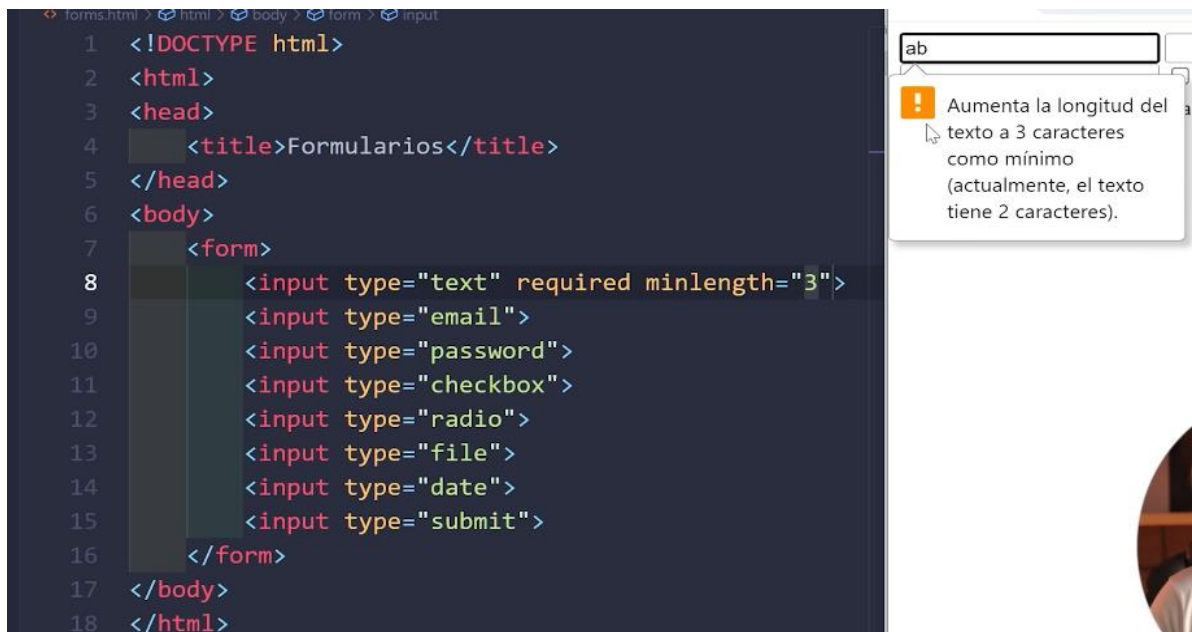
- placeholder= es visual, es decir, es un pequeño texto descriptivo que saldrá y desaparecerá cuando escribamos algo



- value= este atributo nos permite a nosotros que por defecto salga ya algo escrito en ese input, no de manera descriptiva, sino que ya sale como si alguien lo hubiera escrito ósea no es un texto así como invisible como el del placeholder, sino que es un texto firme ya escrito jaja que debemos seleccionar y colocar otro valor en caso de que queramos:



- minlength= es para poner un mínimo de caracteres, por ejemplo no hay nadie que se llame “a” entonces podemos poner que mínimo alguien ponga 3 letras mínimo.



The screenshot shows a code editor with the following HTML code:

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Formularios</title>
5 </head>
6 <body>
7   <form>
8     <input type="text" required minlength="3">
9     <input type="email">
10    <input type="password">
11    <input type="checkbox">
12    <input type="radio">
13    <input type="file">
14    <input type="date">
15    <input type="submit">
16  </form>
17 </body>
18 </html>
```

On the right, a browser window shows a text input field with the text "ab". A tooltip with an orange warning icon says: "Aumenta la longitud del texto a 3 caracteres como mínimo (actualmente, el texto tiene 2 caracteres)."

- maxlength= es para poner un máximo de caracteres, cabe destacar que si ponemos un máximo por ejemplo de 8, cuando escribamos la novena palabra no va a aparecer se va a bloquear en la octava palabra.

CSS

Debemos tener en cuenta que css es un lenguaje de hoja de estilos en cascada, ¿cómo así?, en el sentido de que si nosotros ponemos un class o un id a una misma etiqueta de un párrafo y luego nos vamos al css y le colocamos un color a ese párrafo, tanto un color por el llamado de la clase como por el llamado del id hay un choque de directivas, el párrafo no va a saber qué color elegir, y allí es donde css tiene importancia, ya que si a el párrafo se le está mandando a poner color en dos directrices distintas, el navegador va a seleccionar el color en base a prioridades, el id está por encima del class, por lo cual el navegador seleccionara el color que pusimos al id, y por ejemplo si tenemos un class="hola adios" y en el css hacemos llamado a colocar un color a ese párrafo tanto por hola "como" por "adios" son de la misma clase ninguno está por encima del otro, aquí es cuando funciona muy bien la cascada, ya que css decidirá que el color es el del último llamado que se haya hecho en ese archivo style.css, es decir, si nosotros pusimos lo siguiente dentro del style.css:

```
.hola {  
    color: red;  
}  
  
.adios {  
    color: aqua;  
}
```

Va a ganar el "adios" y el párrafo va a ser de color aqua ya que esta de ultimo en el código, por eso el nombre de lenguaje en cascada, si una propiedad es utilizada varias veces para la misma etiqueta gana la última propiedad definida dentro del style.css

Por otro lado, hay 3 formas de utilizar estilos css en html:

- en la línea directa sin hacer uso de un archivo.css por separado, es decir, se pone el css dentro del mismo archivo.html:

```
<!-- Estilo lineal en una sola línea-->  
<h1>TITULO</h1>  
<P style="color: red; font-size: 25px;">  
    Lorem, ipsum dolor sit amet consectetur adipisicing elit. Amet eos omnis accusantium ut illum laboriosam officia consectetur veniam fugiat voluptas fugit  
    rem, adipisci voluptatum distinctio autem, sed, dolorum commodi repellendus!  
</P>
```

Así se referenciaría el estilo css dentro de la misma página html, pero esto no lo debemos hacer por dos cosas, la primera, es que es una mala práctica ósea mala programación ya que no debemos mezclar dos lenguajes en un mismo archivo y lo segundo es que si yo tengo una obstrucción en el código o algo en ese estilo no tengo la ruta por separado, es decir, no tengo una ruta de acceso rápida donde yo

diga “aquí están los estilos” sino que están todos perdidos por los archivos donde está el html.

- En el mismo código html pero como una etiqueta en una sesión aparte, lo cual tampoco es buena práctica, ya que seguimos sobrecargando el código html con esos bloques de css que no deberían mezclarse allí, además muchas veces utilizamos muchas propiedades de css y tener un bloque gigantesco de css y luego volver a html es como raro. De igual manera así se ve:

```
<!-- Estilo en etiqueta separada en bloque dentro del mismo html-->
<h1>TITULO</h1>
<p class="parrafo">
  Lorem, ipsum dolor sit amet consectetur adipisicing elit. Amet eos omnis accusantium ut illum laboriosam officia consectetur veniam fugiat voluptas fugit
  rem, adipisci voluptatum distinctio autem, sed, dolorum commodi repellendus!
</p>
<style>
  .parrafo {
    color: rgb(6, 240, 45);
    font-size: 20px;
    font-family: monospace;
  }
</style>
```

- En un archivo aparte, lo cual es la ideal y es la buena practica pro-premium jajaja:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="1style.css">
</head>
```

Aquí indicamos en el head el enlace hacia que? Hacia un archivo css, ese atributo rel significa “relateintion” o algo así jajajaja lo que hace es que el enlace que vamos a seleccionar tiene que ver en relación aaaaaaaa “correcto, una hoja de estilos” para eso ese atributo y luego el href que es como decir “hoja de estilo esta en” y la ruta, luego en un archivo aparte que es el que estamos enlazando en nuestro html ponemos todas las propiedades que queremos modificar haciendo uso de selectores, que son los selectores, son esto:

```

.daniel {
  color: aqua;
}

.daniel {
  color: rgb(199, 211, 27);
}

.daniel {
  color: rgb(162, 9, 233); /*Gana este color ya que es el ultimo, aunque aplico propiedades 3 veces a un mismo elemento html, gana este porque es el ultimo
que esta a ese elemetno */
}

#daniel {
  color: blueviolet;
}

.alejandro p {
  color: brown;
  background-color: black;
}

#a {
  color: chartreuse;
}

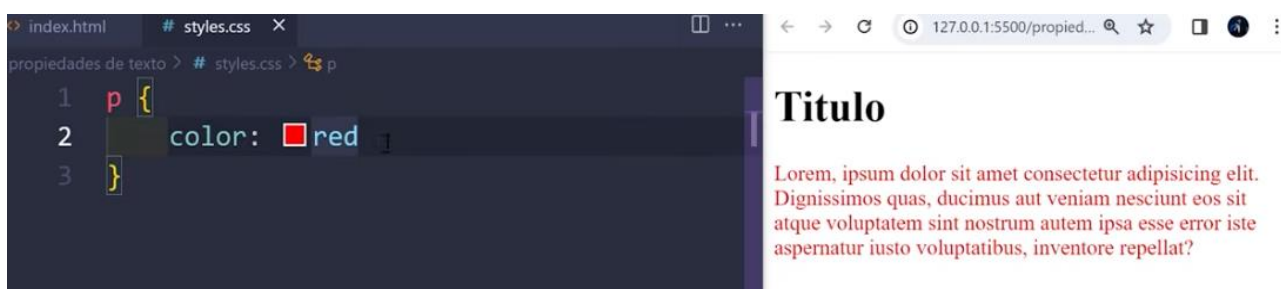
```

Yo porque ya se cómo funcionan un poco los selectores, por ello vemos varias cosas en la foto y surge que cuando trabajamos con CSS podemos hacer uso ya sea de propiedades que vamos a aplicar a todo un conjunto de etiquetas iguales en el index.html, es decir, que a todos los párrafos de nuestro HTML le quede un solo color, o podemos hacer uso de clases que es para aislar una etiqueta de nuestro HTML y dentro del style.css hacemos el llamado a esa clase solo para modificarla a ella sin modificar el resto de etiquetas del código así sean una etiqueta p igual que la que aislamos. El “.” Es para la clase class dentro del HTML, y el “#” es para la clase id dentro del HTML, ósea es para hacer llamado a esas clases, solo que una utiliza un punto y la otra un numeral para que de esta forma el CSS sepa que es una class y el otro es un id. Jaja, por cierto, hay otra etiqueta en HTML que se llama span es como una clase para aislar pero no me convence ajajajaja. Y además debemos tener en cuenta que debemos seleccionar un nombre acorde a las clases ya sea un class o un id, ya que si otro programador lee el código y nosotros le ponemos por nombre a la class o al id “jajajXD” el programador al leer eso diría “QUE MIERDAS QUISO DECIR ESTE LOCO”, así que ojo cuidado con eso.

PROPIEDADES DE TEXTO:

- color:

Nos coloca color a los textos o a elementos de HTML, una cosa importante a destacar es que cuando uno en un archivo style.css le coloca el color a un párrafo o texto nosotros utilizamos la propiedad “color”, pero ojo nosotros seleccionamos el valor de esa propiedad así:



Pero no es bueno trabajar con nombre de colores en las propiedades del css, ya que ese “red” en otros navegadores puede que no lo reconozca y que el navegador le coloque otro color o uno similar, mas adelante trabajaremos con colores absolutos hexadecimales los cuales serán igual en todos los navegadores.

- font-family:

Esta propiedad de texto nos permite a nosotros establecer un tipo de letra asi como en Word, cabe destacar que podemos poner el valor de la propiedad, ósea el nombre de la letra ya sea en comillas o sin comillas, solo que debemos utilizar siempre comillas cuando el nombre de la letra tenga mas de una palabra con espacios blancos entre medio, así:

```
p {  
  color: red;  
  font-family: 'Lucida Sans'  
}
```

Titulo

Lorem, ipsum dolor sit amet consectetur adipisicing elit. Dignissimos quas, ducimus aut veniam nesciunt eos sit atque voluptatem sint nostrum autem ipsa esse error iste aspernatur iusto voluptatibus, inventore repellat?

O cuando es una sola palabra:

```
p {  
  color: red;  
  font-family: tahoma  
}
```

Titulo

Lorem, ipsum dolor sit amet consectetur adipisicing elit. Dignissimos quas, ducimus aut veniam nesciunt eos sit atque voluptatem sint nostrum autem ipsa esse error iste aspernatur iusto voluptatibus, inventore repellat?

Cabe destacar que no es necesario terminar con un “;” (punto y coma) en la ultima propiedad del selector, obviamente es buena practica terminar con un punto y coma asi sea la última propiedad que se le va a signar a ese selector, porque no sabemos cuando en el futuro necesitemos colocar otra propiedad y ya tenemos el ; ahí puesto jaja

También podemos hacer una cosa interesante, no si se aplica a todo, pero podemos colocar una propiedad por ejemplo con el “font-family” y colocarle dos valores, ósea, dos tipos de letras, por si de repente la primera letra está mal escrita pase a la segunda y así sucesivamente hasta que encuentra una bien:

```
p {  
  color: red;  
  font-family: sans-serif, tahoma, arial  
}
```

Titulo

Lorem, ipsum
elit. Dignissim
eos sit atque v
esse error iste
inventore repe

Y así podemos poner mas un valor, cabe destacar que trate de hacer lo mismo con los colores jajajajajkajaja y no sirvió, ósea puse mal un nombre luego una “,” seguido otro nombre de otro color, y cuando voy a ver en el navegador no se pone ningún color se queda en negro, y cuando quito el segundo nombre de letra ahí si agarra jajajajaja, por los momentos esto sirve solo con el font-famlily

- font-size:

Esta propiedad de texto es para el tamaño en las letras, debemos tener presente que poner 16px no nos servirá de nada ya que por lo general la mayoría de navegadores le ponen 16px de tamaño por defecto a las letras o textos, ya en 13px hacia abajo o de 17px en adelante si veremos cambios

```
font-size: 13px;
```

- font-weight:

Esta propiedad lo que hace es ponernos mas anchura a la letra, inclusive el modo “negrita” lo declaramos con esta propiedad, ya que en html para poder poner de una en “negrita” la letra teníamos que utilizar la etiqueta , en css solo ponemos esta propiedad font-weight con el valor “bold”

```
font-weight: bold;
```

Cabe destacar que si nosotros solo necesitamos marcar una letra de un párrafo en negrita no es buena práctica poner en el párrafo esto:

```
<h1>Titulo</h1>
<p>
  Lorem, <b>ipsum</b> dolor sit amet consectetur adipisicing
  elit. Dignissimos quas, ducimus aut veniam nesciunt eos sit
  atque voluptatem sint nostrum
```

Titulo

Lorem, **ipsum** dolor sit amet consectetur adipisicing elit. Dignissimos quas, ducimus aut veniam nesciunt eos sit atque voluptatem sint nostrum autem ipsa esse error iste aspernatur iusto voluptatibus, inventore repellat?

Es mejor por buena práctica utilizar la etiqueta strong en vez de “b” y eso porque?, ya que los navegadores le van a dar mucha importancia a esta etiqueta, en cambio a “b” no mucha importancia, pero las dos hacen lo mismo

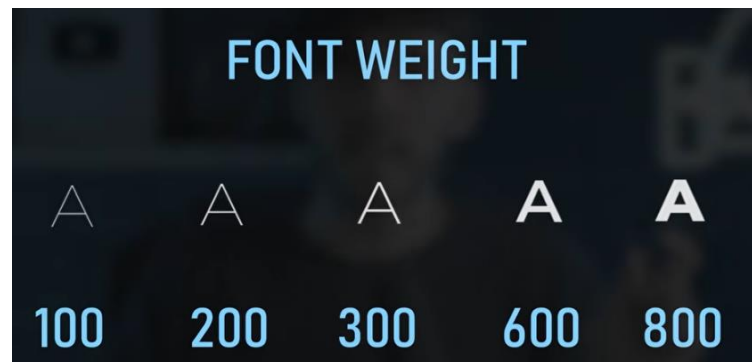
```
<h1>Titulo</h1>
<p>
  Lorem, <strong>ipsum</strong> dolor sit amet consectetur
  adipisicing elit. Dignissimos quas, ducimus aut veniam
```

Titulo

Lorem, **ipsum** dolor sit amet consectetur adipisicing elit. Dignissimos quas, ducimus aut veniam nesciunt eos sit atque voluptatem sint nostrum autem ipsa esse error iste aspernatur iusto voluptatibus, inventore repellat?

Inclusive podemos hacer uso de la etiqueta “span” que es como un class así pero mínimo, pero ese no se utilizarlo mucho (Buscarlo a futuro para practica)

Ahora si continuando con este atributo de Font-weight debemos tener en cuenta que aparte de ponerlo en negrita también contamos con poner valores para ajustarlo a nuestra necesidad:



Pero qué pasa con esto, surge que por ejemplo yo “Daniel Rosario” cree una tipografía y seleccione un weight para la misma, cuando yo me vaya al código y utilice esa tipografía y le trate de ajustar un weight, si el valor del weight esta por debajo de lo que yo le puse cuando cree esta tipografía al ver en el navegador no veremos diferencia de tamaño, solo veremos la diferencia de tamaño al pasar por encima del weight que nosotros establecimos a la tipografía que creamos:

```
p {  
  color: red;  
  font-family: sans-serif;  
  font-size: 13px;  
  font-weight: 100  
}
```

Titulo

Lorem, ipsum dolor sit amet consectetur adipisicing elit. Dignissimos quas, ducimus aut veniam nesciunt eos sit atque voluptatem sint nostrum autem ipsa esse error iste aspernatur iusto voluptatibus, inventore repellat?

```
p {  
  color: red;  
  font-family: sans-serif;  
  font-size: 13px;  
  font-weight: 300  
}
```

Titulo

Lorem, ipsum dolor sit amet consectetur adipisicing elit. Dignissimos quas, ducimus aut veniam nesciunt eos sit atque voluptatem sint nostrum autem ipsa esse error iste aspernatur iusto voluptatibus, inventore repellat?

```
p {  
  color: red;  
  font-family: sans-serif;  
  font-size: 13px;  
  font-weight: 500  
}
```

Titulo

Lorem, ipsum dolor sit amet consectetur adipisicing elit. Dignissimos quas, ducimus aut veniam nesciunt eos sit atque voluptatem sint nostrum autem ipsa esse error iste aspernatur iusto voluptatibus, inventore repellat?

Vemos que el grosor de la letra se mantiene igual, solo vemos diferencia para este tipo de letra en 700 aquí si vemos diferencia:



Y esto se debe a que, a que el creador de ese tipo de letra sans-serif cuando la creo solo definió sus valores en 600m, los 100 200 300 400 500 tienen el mismo finito porque no las definió por separado, sino que tienen lo mismo, y ahora nos preguntaremos, ¿Qué pasa con 800 y 900? Para esta misma tipografía el creador le puso lo mismo, ósea 800 y 900 es lo mismo, es decir, del 100 al 600 es finita, el 700 medio grosor, y 800 y 900 super gruesa, cabe resaltar que como dije esto lo define el autor y es diferente para cada letra dependiendo de lo que le definió su creador, puede haber letras que cada valor tiene un grosor distinto porque si le definieron valor a 100 2000 300 400.... así por separado

Ahora bien, ¿cuál es la diferencia de poner bold a poner un valor?, la diferencia es que bold es como escribir 700 de valor, kajajajajaj ósea bold es igual a 700

A parte de poner bold, también esta bolder, este atributo no es como bold que es fijo en valor 700, sino que bolder busca un escalón mas arriba de lo establecido, si la tipografía ya en 700 es su máximo grosor, bolder se ubica un poco mas arriba y la pone un poquito más gruesa, ósea trata de ir un escalón mas arriba así no exista ese escalón jajajaj

Extensiones instaladas en vscode:

